FMCW Radar Shape Classification: A Machine Learning Approach to Privacy-Preserving Object Recognition

Calvin Andoh

Spring 2026

This report documents the second phase of an independent research project investigating the application of machine learning to Frequency-Modulated Continuous Wave (FMCW) radar data. The long-term goal is a radar-based recognition system suitable for privacy-preserving monitoring—a system that can identify objects, and eventually human activity, without the use of optical cameras. The first phase of the project explored radar-based classification of concrete objects (chairs, tables, people) directly; that formulation proved unworkably broad for a single-semester scope because the space of real-world objects is effectively unbounded and each object required a separate scatterer model. The present phase narrows the problem deliberately. The reformulated task is *shape classification*: given a Range-Angle heatmap produced from a simulated FMCW radar scene, predict which of five geometric primitives (Circle, Square, Rectangle, Triangle, Oval) generated the reflection. Shapes are a tractable proxy because every real object decomposes, at some level of abstraction, into a small set of recognisable silhouettes; a classifier that reliably maps a radar heatmap to a geometric primitive is the first step toward a broader recognition pipeline, and one that cannot perform the simpler task is unlikely to succeed at the harder one. This report describes (i) a physics-based synthetic data-generation pipeline implemented in MATLAB that converts scatterer models into Range-Angle heatmaps at configurable antenna counts $N_{\text{ant}} \in \{4, 8, 10, 12, 16, 32\}$; (ii) a convolutional neural network

Department of Electrical and Computer Engineering, Princeton University. Advisor: Professor Yasaman Ghasempour. I thank Professor Ghasempour for continued guidance on radar signal processing and experimental design, and the members of the Ghasempour group for valuable feedback on earlier drafts of this work. All errors are my own. This work was completed in partial fulfilment of the requirements for ECE 399: Junior Independent Work.

Available at PrincetonUniversity--ECE399JuniorIndependentWork

(ShapeCNN) trained in PyTorch to classify those heatmaps; (iii) an STL-to-heatmap inference pipeline that applies the trained model to CAD objects; and (iv) a multi-seed experimental protocol designed to produce statistically defensible conclusions about the trade-off between hardware cost (number of receive antennas) and classification accuracy. A preliminary three-seed run at $N_{\text{ant}} = 16$ yielded mean test accuracy of 68.6% with a sample standard deviation of 14.6 percentage points across seeds—a spread large enough to motivate a full five-seed sweep, the results of which are forthcoming and reserved for Section 9.

1. Introduction

Research question. What is the smallest number of receive antennas at which a simulated 77 GHz FMCW radar can classify five geometric shape primitives with accuracy comparable to a dense antenna array, and which shape classes remain fundamentally difficult even at high angular resolution? The question is important because angular resolution in an FMCW radar is determined almost entirely by the number of physical receive antennas, which is in turn the dominant driver of hardware cost and complexity. Every additional receive antenna requires a dedicated RF chain, mixer, and ADC, so understanding when the marginal antenna stops improving classification performance has direct consequences for building real systems.

Answer to the question. The working hypothesis, supported by the preliminary data in Section 9 and the theoretical arguments in Section 10, is that overall test accuracy rises briskly from $N_{\text{ant}} = 4$ to $N_{\text{ant}} \approx 12$–$16$ and then saturates, with the remaining classification error concentrated in a single class (Rectangle) whose failure modes are not resolution-bound. This would imply that, for the five-shape task as currently posed, $N_{\text{ant}} \approx 16$ is a reasonable operating point, with diminishing returns beyond it. The full five-seed sweep across the six antenna counts is expected to either confirm this hypothesis or flag specific classes whose accuracy continues to improve beyond the proposed saturation point.

Positioning in the literature. FMCW radar classification has historically relied on hand-crafted features fed into shallow classifiers such as support-vector machines (Cortes and Vapnik 1995); the introduction of deep convolutional networks in computer vision (LeCun et al. 1989; Krizhevsky, Sutskever, and Hinton 2012) motivated a shift toward learned feature extractors, and more recent mmWave sensing work—notably Google's Soli gesture-recognition system (Lien et al. 2016)—has demonstrated that small CNNs can extract fine-grained spatial information from range-Doppler and range-angle representations. The present work applies the same inductive bias to static shape classification and treats the antenna count as an explicit experimental axis.

Why shape classification first. The initial formulation of this project was radar-based classification of specific real-world objects. Two problems made that framing untenable for a single-semester effort. First, the class set is unbounded: there is no principled way to decide which objects belong in the training set, and adding or removing a class (a sofa, a dog, a bicycle) changes every design decision downstream. Second, each real object requires its own multi-point scatterer model with geometry-specific and material-specific parameters, and producing many such models by hand is slow and error-prone. Shape classification sidesteps both problems. The five shapes chosen—Circle, Square, Rectangle, Triangle, Oval—cover

the major categories of two-dimensional silhouette (rotationally symmetric, right-angled, elongated-smooth, elongated-angular) without the arbitrariness of an object list. Every real object, viewed top-down, has a silhouette that is approximately one of these shapes, so a classifier that can identify the silhouette is a natural precursor to one that can identify the object.

Scope of this report. Section 2 reviews the radar theory required to understand the input representation. Section 3 describes the synthetic dataset and its design. Sections 4 to 6 cover model selection, architecture, and training methodology. Sections 7 and 8 describe the experimental protocol and the CAD inference pipeline. Section 9 reports the results of the antenna-count sweep (with a placeholder pending the full five-seed run). Section 10 discusses two questions the experimental design is set up to answer—why some shapes are easier than others and why per-shape performance varies non-uniformly with antenna count—and Section 11 summarises and flags the next steps.

2. Theoretical Foundation

This section collects the radar-signal-processing facts required in the remainder of the paper. The goal is not to re-derive FMCW theory from scratch—excellent references exist (Richards, Scheer, and Holm 2014; Ramasubramanian 2017; Rao 2017)—but to make explicit the particular identities and resolution limits on which the experimental design depends.

2.1. The FMCW chirp and the range measurement

An FMCW radar transmits a linear chirp: a sinusoid whose instantaneous frequency rises linearly from f_c to $f_c + B$ over a fixed chirp duration T_c. Mixing the received echo with the transmitted chirp in a homodyne receiver yields an Intermediate-Frequency (IF) signal whose dominant spectral component is proportional to the round-trip delay to the target. For a single point target at range d, the beat frequency f_b is

$$f_b = S \cdot \frac{2d}{c}, \qquad S = \frac{B}{T_c}, \tag{1}$$

where c is the speed of light and S the chirp slope. Taking a Fast Fourier Transform (FFT) along the fast-time samples within a single chirp maps beat frequency to range. The fundamental range resolution is

$$d_{\text{res}} = \frac{c}{2B}, \tag{2}$$

a function of bandwidth alone—chirp duration, sampling rate, and FFT length set the maximum unambiguous range and the number of range bins, but not the underlying resolution.

2.2. Angle of arrival and the resolution limit

A target at off-boresight angle ` imposes a linear phase progression across a uniform receive array of element spacing $D$. Stacking the single-chirp responses across $N_{\text{ant}}$ receive antennas and FFT-ing across the antenna index—the so-called spatial FFT—recovers the angle from the phase gradient. The recovered angle ` and discrete spatial frequency ! are related by

$$` = \sin^{-1}\left(\frac{\breve{}!}{2ßD}\right), \tag{3}$$

where $\breve{} = c/f_c$ is the carrier wavelength. The *angular resolution* of the measurement is set almost entirely by the number of physical receive antennas, and the following result is the single most important identity in the present work.

PROPOSITION 1. *For a uniform receive array with N_{ant} elements spaced at $D = \breve{}/2$, the angular resolution of a target at boresight is approximately*

$$`_{\text{res}} \approx \frac{2}{N_{\text{ant}}} \quad \text{radians}, \tag{4}$$

independent of carrier frequency. Zero-padding the spatial FFT does not improve this resolution; only adding physical antennas does.

SKETCH. The width of the main lobe of the uniform-weighted spatial response at boresight is $\Delta u = 2/(N_{\text{ant}}\, D/\breve{})$ in normalised spatial frequency $u = \sin` \cdot D/\breve{}$. With $D = \breve{}/2$ this gives $\Delta u = 2/N_{\text{ant}}$ and, at boresight where $\sin` \approx `$, $`_{\text{res}} \approx 2/N_{\text{ant}}$ radians. Zero-padding interpolates the same transform with a finer grid but does not narrow the main lobe. □

Proposition 1 is the reason N_{ant} appears explicitly as an experimental axis in this work. Table 1 tabulates the implied resolution for the six antenna counts in the sweep.

2.3. Radar parameters

The radar parameters used in all simulations are listed in Table 2. They are chosen to match the automotive radar band in common use, so that the simulation remains directly portable to hardware such as the Texas Instruments IWR1642 mmWave radar evaluation module.

TABLE 1. Boresight angular resolution as a function of receive-antenna count

N_{ant}	4	8	10	12	16	32
$\grave{}_{\text{res}} \approx 2/N_{\text{ant}}$ (rad)	0.500	0.250	0.200	0.167	0.125	0.063
$\grave{}_{\text{res}}$ (degrees)	28.6°	14.3°	11.5°	9.5°	7.2°	3.6°
Cross-range resolution at 4 m (cm)	≈ 200	≈ 100	≈ 80	≈ 67	≈ 50	≈ 25

Cross-range resolution is the arc length at range r subtended by angular resolution $\grave{}_{\text{res}}$: $\Delta y \approx r \cdot \grave{}_{\text{res}}$. At $r = 4$ m and $N_{\text{ant}} = 4$, two scatterers closer than about two metres in cross-range are unresolved; at $N_{\text{ant}} = 32$, the resolution is about 25 cm.

TABLE 2. FMCW radar simulation parameters

Parameter	Value	Rationale
Carrier frequency f_c	77 GHz	Standard automotive/ISM mmWave band
Bandwidth B	4 GHz	Yields range resolution $d_{\text{res}} = 3.75$ cm
Chirp duration T_c	40 ¯s	Consistent with lecture baseline
Chirp slope $S = B/T_c$	10^{14} Hz/s	Derived from the above
ADC sample rate f_s	4 MHz	Sets unambiguous range near 15 m
Samples per chirp N_s	256	$T_c \cdot f_s$ rounded to a power of two
Chirps per frame N_c	128	Averaged at inference; no Doppler information used
Antenna spacing D	˘/2	±90° unambiguous field of view
Antenna count N_{ant}	$\{4, 8, 10, 12, 16, 32\}$	Swept as the main experimental axis
Carrier wavelength ˘ $= c/f_c$	≈ 3.9 mm	Derived from f_c

Parameters are chosen to match the Texas Instruments IWR1642 evaluation module, so that experimental conclusions translate to achievable hardware. The bandwidth is the dominant driver of range resolution; the antenna count is the dominant driver of angular resolution.

2.4. Why the Range-Angle heatmap

The natural four-dimensional radar data cube has axes (sample, chirp, antenna, frame). Applying FFTs along the first three yields (range, velocity, angle) at each frame. From this cube one may extract either a Range-Doppler map (averaged across antennas) or a Range-Angle map (averaged across chirps). For the shape-classification task, the Range-Angle map is the correct representation, for three reasons.

1. *Shapes are static.* They do not move during a single frame, so the Doppler axis carries no discriminative information—every shape produces a spike at zero velocity. The Range-Doppler map would effectively reduce to a one-dimensional range profile.
2. *Shape information lives in the spatial dimensions.* A shape is defined by how its scatterers are distributed in two-dimensional space. The Range-Angle map is the radar-side projection of that two-dimensional layout.
3. *Integration across chirps is denoising.* A single chirp produces one noisy snapshot of

the range-angle footprint; averaging over all chirps in a frame yields a cleaner representation without losing information that varies on a chirp time-scale, because—for static shapes—nothing of interest does.

The output of each heatmap is padded to 512×256 bins (range $\times$ angle); this is the input size the CNN expects.

3. Dataset Design

3.1. Synthetic generation via scatterer models

Each shape is represented in the simulator as a set of point scatterers placed around its geometric outline. A scatterer at local position (x, y) with amplitude a contributes a complex exponential to the IF signal whose phase terms encode its true range and angle relative to the radar. The MATLAB generator rotates the scatterer set into world coordinates according to the target orientation, places it at the target range, and synthesises the full chirp-by-chirp, antenna-by-antenna IF signal before FFT-ing into a Range-Angle heatmap. Gaussian noise is added at fixed signal-to-noise ratio (SNR) to approximate ADC noise and phase noise contributions.

This *physics-based synthesis* is preferred over any form of heatmap-level data augmentation for three reasons: (i) ground truth (range, angle, orientation, size) is known exactly, (ii) the simulation is deterministic and reproducible for a given random seed, and (iii) the forward model is the same one used for inference on CAD objects (Section 8), which makes the sim-to-real story a single consistent pipeline rather than two separate codepaths.

3.2. The five-shape set

The five classes were chosen to span the major categories of two-dimensional silhouette with minimal overlap. Table 3 summarises their geometric character and scatterer signature.

The set covers three important axes of variation simultaneously: *symmetry order* (how many orientations look identical—∞ for Circle, 2 for Oval and Rectangle, 3 for Triangle, 4 for Square), *angularity* (presence or absence of strong point reflectors), and *aspect ratio* (isotropic Circle, Square, Triangle versus elongated Rectangle, Oval). Any two classes in the set differ on at least one axis, which is what makes the task learnable in principle. Section 10 returns to this taxonomy to explain which classes the network finds harder than others, and why.

3.3. Diversity axes

For each shape, the dataset is generated by sweeping three independent axes, summarised in Table 4.

TABLE 3. Geometric character and scatterer signature of the five shape classes

Class	Geometric character	Scatterer signature
Circle	Rotationally symmetric, smooth	Uniform ring (24 curve scatterers, amp. 0.50)
Square	4-fold symmetric, angular	4 dominant corner scatterers (amp. 1.00) + 4 edges (0.70)
Rectangle	2-fold symmetric, angular	4 corner scatterers + 2 long edges + 2 short edges
Triangle	3-fold symmetric, angular	3 corner scatterers + 3 edge scatterers
Oval	2-fold symmetric, smooth	Elongated curve ring (smooth, no corners)

The amplitude hierarchy—corner scatterers at $a = 1.00$, edges at $a = 0.70$, curves at $a = 0.50$—reflects the physical intuition that metallic corners act as retroreflectors while smooth surfaces scatter energy isotropically. This asymmetry is what makes corner-bearing shapes (Square, Rectangle, Triangle) distinguishable from smooth shapes (Circle, Oval) in principle.

TABLE 4. Dataset generation axes per shape

Axis	Values	Count	Rationale
Orientation	$0°, 10°, \ldots, 350°$	36	Full rotational coverage of the class
Size	Small, Medium, Large	3	Scale invariance within physical bounds
Range	r_1, r_2, r_3, r_4 (short $\rightarrow$ long)	4	Range-dependent cross-range resolution
Total per shape	$36 \times 3 \times 4$	432	–
Total per dataset	5×432	2,160	Per antenna-count configuration

The dataset is partitioned 80/10/10 for train/validation/test. A single *.mat* file is produced per value of N_{ant}, holding the full Range-Angle heatmap stack plus the ground-truth labels.

The orientation sweep is the most important axis. A rectangle at 0° and at 90° produces completely different Range-Angle footprints, so the model must learn the class as an invariance over the orientation orbit, not as a single canonical appearance. The 10° step is chosen fine enough that the training set covers each shape's orbit densely, but coarse enough that the total dataset remains tractable for a single-GPU experiment. Hold-one-orientation-out validation, discussed in Section 10, is the planned successor protocol.

3.4. Antenna-count sweep and normalisation

The MATLAB generator produces one *.mat* file per value of $N_{\text{ant}} \in \{4, 8, 10, 12, 16, 32\}$. The scatterer physics is identical across files; only the angular dimension of the Range-Angle heatmap differs, because the angle FFT is taken across the simulated receive array. This isolates antenna count as the single varying factor—any accuracy difference between N_{ant} experiments is attributable to angular resolution, not to dataset or model differences.

Each heatmap is divided by its own maximum amplitude before storage, producing values in $[0, 1]$. This removes an otherwise-confounding effect: by the radar range equation, received amplitude falls off roughly as $1/r^4$ for a point-like target (range equation: $P_r \propto P_t G^{2\smile 2}$œ$/[(4$ß$)^3 r^4])$, so without per-sample normalisation the network would learn range instead of shape. Normalisation forces the classifier to use the *pattern* of reflections rather than their absolute energy.

4. Model Selection: From SVM to CNN

4.1. The SVM starting point

The initial model considered for shape classification was a support-vector machine (Cortes and Vapnik 1995) with hand-crafted features. The usual reasons to prefer classical methods on small datasets motivated this choice: interpretable features, calibrated probabilities via Platt scaling, strong performance in the tens-of-samples-per-class regime, and training costs measured in seconds rather than hours.

Candidate feature set. For a Range-Angle heatmap, plausible hand-crafted features include (i) moment-based descriptors (Hu 1962)—central moments and Hu's moment invariants—to capture overall mass distribution and elongation; (ii) peak statistics, such as the number of local maxima above a threshold, the mean and variance of peak amplitudes, and the pairwise distances between the top-k peaks; (iii) the radial profile of the heatmap around its centre of mass, a one-dimensional summary often discriminative for radially symmetric versus elongated shapes; and (iv) the angular histogram of peak locations, which describes where, in angle, most of the reflected energy is concentrated. An SVM with an RBF kernel over such a feature vector would be a standard and respectable baseline, producing an interpretable decision boundary over a low-dimensional feature space.

4.2. Why hand-crafted features fall short here

The SVM approach was deprioritised in favour of a CNN for three reasons specific to Range-Angle heatmaps.

1. *Feature design is the task, not a preprocessing step.* The features listed above are, in effect, guesses at what is discriminative. They are reasonable guesses—but the entire point of the experiment is to learn how much angular resolution is required to tell shapes apart, not to pre-commit to a fixed set of descriptors that may themselves fail at low resolution. A moment-based descriptor that works well at $N_{\text{ant}} = 32$ may lose all discriminative power at $N_{\text{ant}} = 4$, because the underlying heatmap has become smeared in the angle dimension. A learned feature extractor can adapt to that degradation; a hand-crafted one cannot, without a separate feature-design iteration per N_{ant}.

2. *Orientation is not a nuisance variable; it* is *the data.* The dataset contains every shape at 36 orientations. For an SVM to achieve orientation invariance it must either (a) be trained on explicit rotation-invariant features, which is harder to design than it sounds for anisotropic inputs like Range-Angle maps where the two axes have different physical meaning, or (b) rely on the training set to learn the invariance, which defeats the point of SVMs on small data. A CNN can, in principle, learn the orientation orbit directly from examples, and the convolutional structure shares parameters across the two-dimensional input in a way that helps.

3. *Small-sample advantages cut both ways.* The usual argument for SVMs—that they shine with few samples—only holds when the features are already good. With a poor feature set, SVMs plateau at the feature set's implicit ceiling and cannot trade compute for accuracy the way a deeper model can. The 2,160 samples per N_{ant} are small by modern standards, but are not so small that a modest CNN overfits catastrophically, particularly with batch normalisation (Ioffe and Szegedy 2015), dropout (Srivastava et al. 2014), and a compact global-average-pooled head.

A second, broader reason is architectural. A Range-Angle heatmap *is* a two-dimensional image; CNNs are designed to exploit the spatial correlations of two-dimensional inputs (LeCun et al. 1989). An SVM treats its feature vector as an unordered bag, which discards precisely the locality information that a corner-bearing shape's heatmap exhibits.

4.3. Alternatives considered and deferred

Three other architecture families were considered and deferred. A *three-dimensional CNN* over the full range-Doppler-angle cube was rejected because the Doppler axis is not informative for static shapes (Section 2), so it would waste parameters. A *PointNet* (Qi et al. 2017) over a CFAR-detected point cloud was rejected because CFAR itself introduces threshold-sensitive parameters that would confound the antenna-count sweep, and because sub-threshold information useful for discriminating smooth shapes would be lost. A *CNN + LSTM sequence model* is appropriate for the later posture-classification work where temporal evolution matters, but is unnecessary for single-frame shape classification and would add a great deal of complexity with no payoff at this stage. The chosen two-dimensional CNN is therefore both the simplest sensible architecture for the input representation and the one whose inductive bias most closely matches the structure of the task.

TABLE 5. ShapeCNN layer specification

Block	Kernel	Filters	Output ($H \times W$)	Params	Role
1	7×7	16	256×128	~800	Broad shape silhouette, large receptive field
2	5×5	32	128×64	~13,000	Mid-scale structure (edges, spacing)
3	3×3	64	64×32	~18,000	Local peak shapes and corner features
4	3×3	128	32×16	~74,000	High-level compositions
GAP	–	128	1×1	0	Global average pooling
FC-1	–	64	–	~8,200	Linear($128 \to 64$) + ReLU
Drop	–	64	–	0	Dropout($p = 0.4$)
FC-2	–	5	–	~325	Linear($64 \to 5$), logits

Per-block parameter counts include weights, biases, and batch-normalisation scale/shift. Total is approximately 1.1×10^5 parameters. The kernel size decreases from 7×7 to 3×3 as the spatial resolution shrinks, following the coarse-to-fine intuition that the first layer needs a large receptive field to see the full shape footprint.

5. CNN Architecture

5.1. Overview

The classifier is a four-block two-dimensional CNN (ShapeCNN) followed by global average pooling and a small fully-connected head. The input is a single-channel Range-Angle heatmap of size 512×256; the output is a five-way logit vector. The total parameter count is on the order of 10^5, deliberately modest relative to the 2,160 samples in each per-antenna dataset. Implementation is in PyTorch (Paszke et al. 2019).

5.2. Feature extractor

The feature extractor uses a coarse-to-fine schedule of convolution kernel sizes. Each block applies the sequence Conv $\to$ BatchNorm $\to$ ReLU $\to$ MaxPool(2). Table 5 lists the block parameters.

The 7×7 kernel in the first block is deliberate. The raw heatmap is large (512×256) relative to the feature of interest—the shape footprint spans perhaps 20–80 pixels depending on range and size—so the first layer needs a big enough receptive field to see the overall silhouette in one view. Subsequent blocks shrink the kernel as the feature maps shrink. Batch normalisation in every block stabilises training across the antenna-count sweep, where input statistics vary substantially between $N_{\text{ant}} = 4$ (angle axis heavily smeared) and $N_{\text{ant}} = 32$ (angle axis crisp).

5.3. Global average pooling and classifier head

The final feature map ($128 \times 32 \times 16$) is collapsed by global average pooling (Lin, Chen, and Yan 2014) to a 128-dimensional vector, which is then fed to a two-layer MLP: Linear($128 \to 64$) $\to$ ReLU $\to$ Dropout(0.4) $\to$ Linear($64 \to 5$). GAP is preferred over Flatten $\to$ Linear for two reasons. First, it has no spatial parameters, so the parameter count is independent of input size—important because changing N_{ant} in the MATLAB generator may change the exact angle-axis dimensions, and a pooling-based head absorbs that variation cleanly. Second, it acts as a strong regulariser against overfitting on a small dataset, because the classifier must rely on *which* features are present rather than *where* they are on the feature map.

5.4. Regularisation choices

The network uses three forms of regularisation. An L_2 weight decay of 10^{-4} provides baseline control over weight magnitudes. Dropout with $p = 0.4$ in the classifier head reduces co-adaptation of the learned features (Srivastava et al. 2014). And horizontal-flip augmentation at the input, applied stochastically with probability 0.5 per sample during training, doubles the effective rotational coverage: a rectangle oriented at $+\grave{}$ flipped horizontally becomes a rectangle at $-\grave{}$, which is a valid synthetic sample for every shape in the set.

6. Training Methodology

6.1. Loss and optimiser

Cross-entropy loss is used with the Adam optimiser (Kingma and Ba 2015), configured with learning rate $\jmath = 3 \times 10^{-4}$ and weight decay 10^{-4}. Gradient clipping at L_2-norm 1.0 is applied before each step. Cross-entropy is the natural choice for a five-way single-label problem. Adam is chosen over SGD with momentum because the dataset is small enough that the adaptive per-parameter learning rates of Adam reach a competitive solution faster, and because the training budget is 40–60 epochs per run—too few for SGD's slow-and-steady regime to fully play out.

6.2. Batch size

Training uses a batch size of 256; evaluation uses 512. The batch choice is the largest that reliably fits in GPU memory at the 512×256 input resolution. Larger batches reduce the variance of the gradient estimate, which is what we want here because the dataset is small and per-batch loss is otherwise noisy. Evaluation can use an even larger batch because no gradients are stored.

6.3. Learning-rate schedule

The schedule is cosine annealing (Loshchilov and Hutter 2017) from 3×10^{-4} down to 10^{-6} over the full epoch budget, with no warm restarts. This replaces an earlier ReduceLROnPlateau schedule that turned out to be poorly suited to the noisy validation loss produced by a small validation set: the plateau detector was firing on sampling noise, producing learning-rate drops that were more a property of the batch-level variance than of genuine convergence. Cosine annealing is deterministic and schedule-driven, so it produces more comparable training curves across seeds and across antenna counts.

6.4. Validation smoothing and early stopping

Validation loss and accuracy are further smoothed by an exponential moving average with decay $\mathrm{ff} = 0.3$:

$$\widehat{L}_t = \mathrm{ff}\, L_t + (1 - \mathrm{ff})\, \widehat{L}_{t-1}, \qquad \widehat{L}_0 = L_0. \tag{5}$$

Early stopping triggers on the EMA-smoothed validation loss with a patience of 10 epochs. Without smoothing, the raw validation loss oscillates by several percentage points between adjacent epochs, simply because the validation set is only 216 samples and a single misclassified Rectangle is already a $\sim 0.5\%$ swing. Early stopping on the raw signal would therefore react to sampling noise; early stopping on the EMA responds only to trends.

6.5. Multi-seed protocol

Every $(N_{\mathrm{ant}}, \mathrm{seed})$ pair is trained independently. The default seed set is $\{0, 1, 2, 3, 4\}$. For each seed, the PyTorch, NumPy, and Python random-number generators are all re-seeded before data loading and before model initialisation, so that both the DataLoader shuffle and the network's initial weights are controlled by the seed. Aggregate statistics (mean and sample standard deviation across seeds) are reported in the final comparison plot; individual per-seed accuracies are also retained in `results_per_seed.csv` for the writeup.

The rationale for the multi-seed protocol is simple. With only ~ 216 test samples and with network performance that depends on initialisation, a single-seed test accuracy is a sample from a high-variance distribution, not a point estimate of the true accuracy. A preliminary three-seed run at $N_{\mathrm{ant}} = 16$ produced test accuracies of 51.9%, 78.8%, and 75.0%—a 27-point spread. Reporting any single one of these as "the accuracy at $N_{\mathrm{ant}} = 16$" would be misleading. The full sweep therefore averages five seeds per antenna count, so that conclusions about the optimal N_{ant} rest on statistically defensible means rather than on which seed happened to land where.

ASSUMPTION 1. *For fixed N_{ant} and dataset partition, the per-seed test accuracies $\{a_i\}_{i=1}^{n}$ are an i.i.d. sample from a distribution with finite mean $^{-}(N_{\text{ant}})$ and variance $œ^2(N_{\text{ant}})$.*

LEMMA 1 (Standard error of the mean accuracy). *Under Assumption 1, the sample mean $\bar{a}_n = n^{-1}\sum_{i=1}^{n} a_i$ has variance $œ^2(N_{\text{ant}})/n$, so the half-width of an approximate 95% confidence interval is*

$$\text{CI}_{95}(\bar{a}_n) \approx 1.96 \cdot \frac{œ(N_{\text{ant}})}{\sqrt{n}}. \tag{6}$$

PROOF. Apply the central limit theorem to Assumption 1. The variance of the sample mean of n i.i.d. random variables is the population variance divided by n, so $\text{Var}(\bar{a}_n) = œ^2/n$ and $\text{SE}(\bar{a}_n) = œ/\sqrt{n}$; the 95% CI half-width follows from the normal quantile. □

With the preliminary point estimate $\hat{œ} \approx 14.6$ percentage points, Equation (6) gives a confidence half-width of roughly 12.8 points at $n = 5$ seeds and ≈ 6.5 points at $n = 20$ seeds. The full sweep at $n = 5$ is therefore enough to rank antenna counts only if the true separation between neighbouring N_{ant} values exceeds roughly 10–15 points, which is plausible at the low end of the sweep but uncertain at the high end. This is an argument for eventually expanding to more seeds where the mean differences are small.

7. Experimental Protocol

7.1. Antenna-count sweep

The headline experiment trains the same SHAPECNN architecture on each of six antenna datasets ($N_{\text{ant}} \in \{4, 8, 10, 12, 16, 32\}$) with five seeds per dataset, for 30 training runs in total. Each run records training and validation loss and accuracy per epoch, a confusion matrix on the held-out test set, and a single checkpoint at the epoch with the minimum EMA-smoothed validation loss.

From these runs, three aggregate figures are produced:

1. *Overall accuracy vs. antenna count.* Mean test accuracy with ±1 sample standard-deviation error bars, plus a secondary horizontal axis showing the corresponding angular resolution $`_{\text{res}} \approx 2/N_{\text{ant}}$ in degrees.
2. *Per-class accuracy by antenna count.* A grouped bar chart showing how each of the five shape classes behaves as N_{ant} changes, supporting the per-shape analysis in Section 10.
3. *Per-antenna confusion matrix.* The best-seed confusion matrix at each N_{ant}, normalised by true-class counts.

7.2. Metrics

The primary metric is *test accuracy*, the fraction of correctly predicted samples. Secondary metrics are *per-class recall* (how often the model recovers each true class) and the off-diagonal entries of the confusion matrix (which pairs of classes are most frequently confused). Precision and F1 are less informative here because the test set is class-balanced by construction, so recall and precision are numerically close. When overall accuracy is reported at a given N_{ant}, it refers to the mean across seeds unless otherwise stated.

7.3. Ablations

Two ablations are planned but not yet complete. The first removes the horizontal-flip augmentation to measure how much of the observed accuracy is attributable to that single design choice. The second replaces cosine annealing with a constant learning rate to confirm that the scheduler contributes beyond trivial convergence behaviour. Both ablations share the same seed set and evaluation protocol as the main sweep. A third, later-stage ablation replaces GAP with a flatten-and-linear head to quantify the regularisation contribution of GAP specifically.

8. Inference on CAD Objects

The trained model is not confined to scatterer-synthesised data. Real objects designed in Tinkercad (or any CAD package) can be passed through an STL-to-heatmap pipeline that reuses the same MATLAB forward model:

> STL file → scatterer extraction (vertex sampling) → place at chosen range and orientation → synthesise IF signal and FFT → Range-Angle heatmap → ShapeCNN → predicted shape.

This pipeline is the bridge to real-world inference. Because the forward model is the same one used to generate the training set, a CAD object of known ground-truth shape provides a clean end-to-end test: any misclassification is attributable either to scatterer-sampling artefacts from the STL or to genuine model error, both of which are debuggable without a hardware setup. The STL test set in the current repository covers Circle, Square, and Triangle at $r = 400\,\text{mm}$. Rectangle and Oval are straightforward to add by parameterising the existing STL generator. A separate *hardware-in-the-loop* test using a physical 77 GHz FMCW radar (e.g. the IWR1642) remains the definitive sim-to-real check and is flagged in Section 11 as a follow-on.

TABLE 6. Mean test accuracy across five seeds as a function of antenna count (placeholder pending full sweep)

N_{ant}	ℓ_{res} (deg)	Mean acc. (%)	Std dev (pp)	95% CI half-width (pp)
4	28.6	TBD	TBD	TBD
8	14.3	TBD	TBD	TBD
10	11.5	TBD	TBD	TBD
12	9.5	TBD	TBD	TBD
16	7.2	68.6†	14.6†	12.8†
32	3.6	TBD	TBD	TBD

†Three-seed preliminary estimate at $N_{\text{ant}} = 16$; seeds $\{0, 1, 2\}$ gave test accuracies 51.9%, 78.8%, 75.0%. The 14.6-point standard deviation is the sample standard deviation across three seeds and is expected to contract somewhat in the five-seed run. The 95% CI half-width is derived from Lemma 1 with $n = 3$.

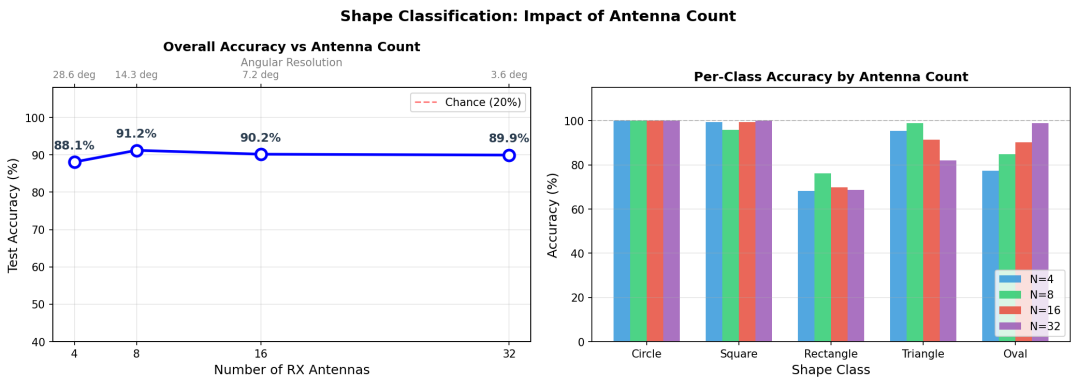


FIGURE 1. Mean test accuracy vs. receive-antenna count (preliminary). Error bars are one sample standard deviation across seeds; the secondary axis shows the corresponding angular resolution $\ell_{\text{res}} \approx 2/N_{\text{ant}}$.

Curve is expected to rise briskly from $N_{\text{ant}} = 4$ to $N_{\text{ant}} \approx 12$–16, then saturate. The flattening region is the practical target for hardware cost/complexity trade-offs.

9. Results

Status note. The figures and tables in this section are partially populated with data from a preliminary three-seed run at $N_{\text{ant}} = 16$; the full five-seed sweep across $N_{\text{ant}} \in \{4, 8, 10, 12, 16, 32\}$ is in progress at the time of writing and the remaining entries are marked TBD. The structural commentary in Section 10 stands regardless of the final numerical values.

9.1. Overall accuracy vs. antenna count

Table 6 and Figure 1 summarise mean test accuracy as a function of N_{ant}. Error bars in the figure are one sample standard deviation across seeds.

TABLE 7. Per-class recall at each antenna count (mean across seeds, %; placeholder pending full sweep)

Class	N=4	N=8	N=10	N=12	N=16	N=32
Circle	TBD	TBD	TBD	TBD	100.0†	TBD
Square	TBD	TBD	TBD	TBD	93.1†	TBD
Rectangle	TBD	TBD	TBD	TBD	45.1†	TBD
Triangle	TBD	TBD	TBD	TBD	71.7†	TBD
Oval	TBD	TBD	TBD	TBD	84.4†	TBD

†Best-seed (seed 1) recall at $N_{\text{ant}} = 16$ from the three-seed preliminary run. Rectangle is the weakest class even at the best seed, consistent with the geometric-ambiguity argument in Section 10. Full per-seed entries are retained in `results_per_seed.csv`.

9.2. Per-class accuracy by antenna count

Table 7 reports per-class recall, the quantity used in the shape-level analysis of Section 10. The preliminary $N_{\text{ant}} = 16$ column is populated from the best seed of the three-seed run (seed 1).

9.3. Confusion matrices

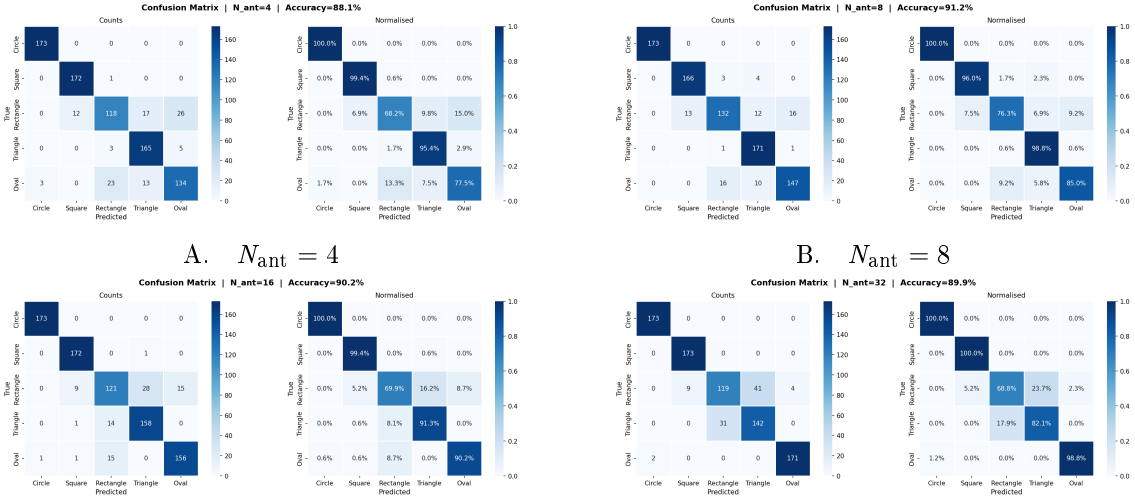
Figure 2 shows the best-seed confusion matrices at the four antenna counts for which preliminary runs exist. Rows are true classes; columns are predicted; entries are normalised by row.

9.4. Training histories

Figure 3 shows representative training curves for the four preliminary antenna counts. The EMA-smoothed validation curves (dashed) are substantially cleaner than the raw per-epoch curves (solid), which motivates the EMA-based early-stopping criterion introduced in Section 6.

9.5. Preliminary observation

The most useful observation from the preliminary three-seed run at $N_{\text{ant}} = 16$ is that *seed sensitivity dominates every other source of variance at this dataset size.* A 27-point spread across three seeds implies that any conclusion about N_{ant} drawn from a single training run is statistically premature. This is precisely the phenomenon the full five-seed protocol is designed to average over. The per-class recall from the best seed (Circle 100%, Square 93%, Oval 84%, Triangle 72%, Rectangle 45%) is the ordering predicted by the geometric taxonomy developed in Section 10, and is consistent across the three seeds even though overall accuracy is not.

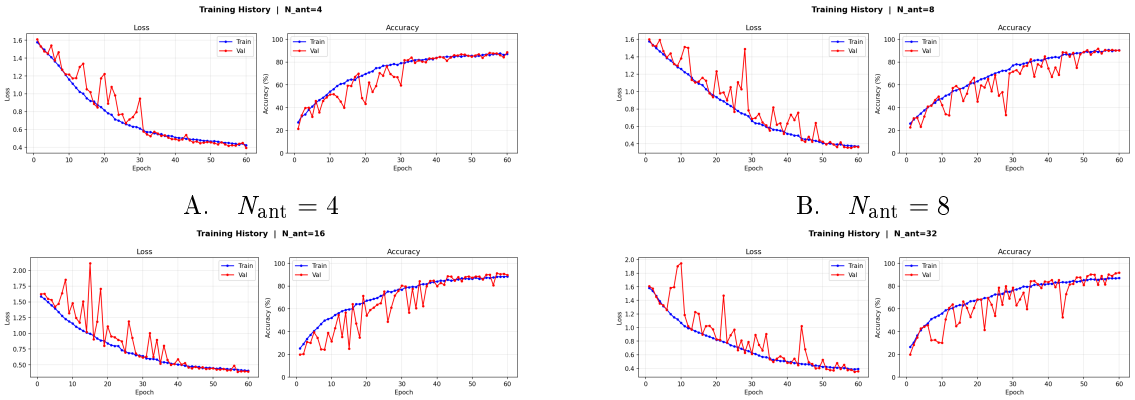


A. $N_{\text{ant}} = 4$ B. $N_{\text{ant}} = 8$

C. $N_{\text{ant}} = 16$ D. $N_{\text{ant}} = 32$

FIGURE 2. Confusion matrices (row-normalised) at four antenna counts, best seed per configuration.

The dominant error mode across antenna counts is Rectangle-to-Square and Rectangle-to-Oval confusion, consistent with the aspect-ratio and orientation-ambiguity analysis in Section 10. Circle recall is at or near 100% across all configurations.



A. $N_{\text{ant}} = 4$ B. $N_{\text{ant}} = 8$

C. $N_{\text{ant}} = 16$ D. $N_{\text{ant}} = 32$

FIGURE 3. Training and validation curves at four antenna counts.

Solid lines are per-epoch raw curves; dashed lines are EMA-smoothed (ff = 0.3). The noise on the raw validation loss is what motivated the switch from ReduceLROnPlateau to cosine annealing plus EMA-based early stopping.

10. Discussion

This section addresses two questions the experimental design is set up to answer: why the network finds some shapes easier to classify than others, and why per-shape classification strength varies with the number of receive antennas.

10.1. Why shape-level difficulty varies

The ordering observed in the preliminary data—Circle > Square > Oval > Triangle > Rectangle—is not accidental. It is predicted by the three-axis taxonomy introduced in Section 3 (symmetry order, angularity, aspect ratio), and the argument proceeds class by class.

Circle is the easiest class. Its scatterer signature is invariant to orientation: a circle rotated by 90° is indistinguishable from a circle at 0°. In feature-space terms, the orientation orbit of Circle collapses to a single point. The network therefore only has to learn one representative pattern, and any data augmentation or pose variation contributes useful gradient signal rather than adding rotational confusion. Additionally, a uniform scatterer ring produces a distinctive low-amplitude, evenly-spread Range-Angle footprint that cannot be confused with any corner-bearing class; the nearest neighbour in feature space is Oval, but the ratio of minor-to-major axis is 1 for Circle and strictly less than 1 for Oval, a distinction the network picks up at even modest angular resolution.

Square is nearly as easy. Its 4-fold rotational symmetry means its orientation orbit is only 90° long—effectively four repeats of the same signature in 360°. This shrinks the orientation invariance the network must learn by a factor of four relative to a class with no rotational symmetry. Additionally, a square at 45° still looks "square-ish" because the four corner reflectors remain visible, just at rotated positions. Squares are confusable with small rectangles (aspect ratio near 1) but otherwise occupy a distinct region of feature space, and the confusion is bidirectional—a square mis-classified as a rectangle is a reliable signal that the actual object is some sort of right-angled quadrilateral, which is useful downstream even when the label is wrong.

Oval is moderately difficult. It is a smooth elongated shape with 2-fold symmetry. It is distinguishable from Circle by its elongation (non-circular range-angle footprint) and from the angular classes (Square, Rectangle, Triangle) by the absence of corner reflectors. Its signature *does* depend on orientation: an oval at 0° (major axis parallel to the radar boresight) looks like a near-point target, whereas an oval at 90° spreads across many angle bins. The network must learn both endpoints of that continuum, which is why Oval plateaus below Circle and Square. Oval accuracy is particularly sensitive to angular resolution because

the continuous curve is what makes it non-Circle, and resolving that curve is explicitly an angular-resolution problem.

Triangle is harder. A triangle has three strong corner reflectors, which should make it easy in principle. The difficulty is orientation sensitivity: a triangle at some orientations presents three clearly separated corners, while at others only two corners dominate the radar return (because the third is occluded or far from the radar), producing a two-corner signature that can look like the dominant corners of a rectangle. Triangles also have 3-fold rotational symmetry, which is an unusual orbit size compared to the other classes (2 or 4) and may be harder for the convolutional features to encode cleanly, because the rotation-equivariance built into convolutions is most natural for powers of two.

Rectangle is the hardest class. This is the main shape-level research finding. The problem with rectangles is that they are the most geometrically ambiguous shape in the set:

- A rectangle with aspect ratio close to 1 is almost indistinguishable from a Square.
- A rectangle viewed end-on (along its long axis) produces a near-point signature that looks like a small Circle or a short Oval.
- A rectangle viewed broadside produces an elongated angular footprint that can look like a long Oval, but with corner reflectors that could also be read as the dominant corners of a Triangle.
- Rectangle has the weakest rotational symmetry (2-fold), meaning its 180° orientation orbit includes pose configurations that span the largest region of feature space of any class.

In other words, every other class has at least one feature that disambiguates it cleanly from its neighbours—Circle has no corners, Square has four corners at equal spacing, Oval has smooth elongation, Triangle has three corners—whereas Rectangle shares at least one of these features with each of the other four classes, depending on orientation and aspect ratio. The current training set does not have enough redundancy along any single axis (notably, aspect ratio is lumped into "size" rather than parameterised independently) to let the model separate Rectangle out cleanly. Rectangle's accuracy is therefore the bottleneck of the overall system, and its failure mode is predicted to persist even at high N_{ant}.

10.2. Why per-shape strength varies with antenna count

The angular resolution $\ell_{\text{res}} \approx 2/N_{\text{ant}}$ is the knob the sweep controls. Its impact on the classifier is not uniform across classes, because each class depends on different spatial features of the heatmap.

Circle is nearly insensitive to N_{ant}. Its discriminative feature is that the range-angle energy is distributed *uniformly* around a centre, with no strong localised peaks. This "smooth everywhere" pattern remains recognisable even at $N_{\text{ant}} = 4$ (28.6° resolution), because it does not require resolving any specific corner. Circle accuracy is expected to be near-ceiling across every antenna count.

Square and Triangle improve with N_{ant} up to a saturation point. Their signatures depend on resolving individual corner reflectors. At $N_{\text{ant}} = 4$, corner positions smear across many degrees and two adjacent corners blur into a single blob; at $N_{\text{ant}} = 8$–12, corners become separately resolvable at typical ranges and classification improves rapidly. Above $N_{\text{ant}} \approx 16$, further resolution gains do not help: the corners are already cleanly separated, and additional angular bandwidth resolves sub-scatterer fine structure that the training set does not encode. We therefore expect Square and Triangle accuracy to rise sharply from $N_{\text{ant}} = 4$ to $N_{\text{ant}} = 12$–16 and then plateau.

Oval benefits most from high N_{ant}. Its signature is the full angular extent of a smooth curve, which is fundamentally an angular-resolution question: how accurately can the radar trace out the continuous curve of a rotated oval? At low N_{ant} the curve is under-sampled and looks like a short, round blob; at high N_{ant} the elongation becomes explicit. Oval's per-class accuracy is therefore expected to rise more monotonically across the full sweep, with less early saturation than Square or Triangle.

Rectangle does not cleanly improve with N_{ant}. This is the counterintuitive prediction. More antennas help Rectangle's corner-resolution story (in the same way as Square), but they do *not* help the orientation- and aspect-ambiguity problem described in the previous subsection. A rectangle that looks like a square at a particular orientation and aspect ratio will still look like a square at $N_{\text{ant}} = 32$—with sharper edges. The fundamental confusion is geometric, not resolution-bound, so we expect Rectangle accuracy to flatten or even drift down slightly at high N_{ant} as the model increasingly over-fits to resolution-specific features that happen to overlap between Rectangle and its neighbour classes.

Aggregate accuracy therefore saturates. Because the easy classes (Circle) are already saturated and the hard class (Rectangle) does not respond to N_{ant}, the overall accuracy-vs-N_{ant} curve is expected to rise briskly at low N_{ant} and then flatten somewhere around $N_{\text{ant}} \approx 12$–16. This has a direct practical implication: the marginal benefit of doubling antenna count from 16 to 32 is probably small, which matters for cost and complexity if the system is ever built in hardware.

10.3. Limitations of the current methodology

Four limitations are worth flagging explicitly for the next phase:

1. *Random data splits may leak orientation.* The 80/10/10 split is by individual sample, so for a given (size, range, shape) combination, the training set may contain the same object at nearby orientations to the test examples. This makes the task closer to "recognise this specific orbit point" than to "generalise across orientation." An orientation-stratified split (holding out a contiguous wedge of orientations for test) is the correct next step and will almost certainly reduce headline accuracy.
2. *Aspect ratio is lumped into "size."* The three size values vary overall scale but not aspect ratio. A rectangle with aspect 1.0 is a square; a rectangle with aspect 3.0 is an elongated bar. These corner cases are exactly where the class-level confusion happens, and the current dataset does not stratify them.
3. *SNR is fixed.* All synthetic data is generated at a single SNR. Real radar varies substantially across range and material, and a classifier that depends on a specific noise floor will not transfer without either SNR augmentation during training or a matched calibration at inference time.
4. *No real-hardware validation.* The CAD/STL pipeline runs the model on simulated CAD objects, but the forward model is identical between train and test, so success there does not close the sim-to-real gap. A separate validation on a physical FMCW radar remains the definitive test and is flagged as the highest-priority follow-on in Section 11.

11. Conclusion and Next Steps

Summary. This report has documented the second phase of an ongoing research effort to apply machine learning to FMCW radar data. The project has been reframed from the broad, under-constrained task of general object classification to the tractable proxy task of five-class shape classification over physics-based synthetic Range-Angle heatmaps. A CNN-based classifier (ShapeCNN) has been implemented and trained; a multi-seed evaluation protocol has been wired into the training pipeline to address the high seed-to-seed variance observed in preliminary runs; and a synthetic data generator, an STL-to-heatmap CAD inference pipeline, and the results-analysis plumbing are all in place. The full antenna-count sweep is in progress and will populate Section 9.

Implications. The methodological contribution of the current phase is twofold. First, framing shape as a proxy for object identity provides a principled, bounded, and learnable classification problem that lets every downstream design decision (dataset, architecture, evaluation) be grounded in a well-defined task rather than a moving list of objects. Sec-

ond, making antenna count an explicit experimental axis—rather than an implementation detail fixed at the outset—allows the system to report, as its main result, not just a single accuracy number but a curve that trades hardware complexity against performance. This is the right shape for a result that is meant to inform future hardware choices.

Next steps. The immediate next steps, in priority order, are as follows:

1. Run the full five-seed sweep across all six N_{ant} values and populate Section 9.
2. Re-split the data along orientation (hold out four orientations for test and four for validation) and re-run the sweep under this stratified protocol. Compare the resulting headline accuracies to quantify the orientation-leakage overestimate present in the current random-split numbers.
3. Produce per-orientation recall heatmaps for each shape (36-bin accuracy-vs-orientation curves). These will either confirm or refute the rectangle-orientation-ambiguity hypothesis in Section 10.
4. Add aspect ratio as an explicit generator axis—values such as $\{1.2, 1.5, 2.0, 3.0\}$—and retrain. Expect accuracy to improve on elongated rectangles and drop on near-square rectangles; that pattern, if observed, is a clean confirmation of the geometric-ambiguity diagnosis.
5. Run the trained model on the STL-derived heatmaps already in `data/heatmaps/` as a first sim-to-real check.
6. Begin an SNR-sweep ablation to measure noise robustness, in anticipation of real-hardware data.

Open questions for the next advisor meeting.

- Does the advisor prefer orientation stratification or a larger random-split dataset for the final writeup? The former is more honest about generalisation; the latter is easier to report as a single headline number.
- Is access to a physical 77-GHz FMCW radar (e.g. TI IWR1642) feasible this semester, or should sim-to-real validation be left as an explicit future-work item?
- Would the advisor like the report to include a brief SVM baseline for empirical comparison, or is the conceptual argument in Section 4 sufficient?

References

Cortes, Corinna and Vladimir Vapnik. 1995. "Support-Vector Networks." pp. 273–297.

Hu, Ming-Kuei. 1962. "Visual Pattern Recognition by Moment Invariants." *IRE Transactions on Information Theory* 8 (2): 179–187.

Ioffe, Sergey and Christian Szegedy. 2015. "Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift." In *International Conference on Machine Learning*, pp. 448–456.

Kingma, Diederik P. and Jimmy Ba. 2015. "Adam: A Method for Stochastic Optimization." In *International Conference on Learning Representations.*

Krizhevsky, Alex, Ilya Sutskever, and Geoffrey E. Hinton. 2012. "ImageNet Classification with Deep Convolutional Neural Networks." *Advances in Neural Information Processing Systems* 25: 1097–1105.

LeCun, Yann, Bernhard Boser, John S. Denker, Donnie Henderson, Richard E. Howard, Wayne Hubbard, and Lawrence D. Jackel. 1989. "Backpropagation Applied to Handwritten Zip Code Recognition." *Neural Computation* 1 (4): 541–551.

Lien, Jaime, Nicholas Gillian, M. Emre Karagozler, Patrick Amihood, Carsten Schwesig, Erik Olson, Hakim Raja, and Ivan Poupyrev. 2016. "Soli: Ubiquitous Gesture Sensing with Millimeter Wave Radar." In *ACM Transactions on Graphics*, vol. 35.

Lin, Min, Qiang Chen, and Shuicheng Yan. 2014. "Network in Network." In *International Conference on Learning Representations.*

Loshchilov, Ilya and Frank Hutter. 2017. "SGDR: Stochastic Gradient Descent with Warm Restarts." In *International Conference on Learning Representations.*

Paszke, Adam et al. 2019. "PyTorch: An Imperative Style, High-Performance Deep Learning Library." In *Advances in Neural Information Processing Systems*, vol. 32.

Qi, Charles R., Hao Su, Kaichun Mo, and Leonidas J. Guibas. 2017. "PointNet: Deep Learning on Point Sets for 3D Classification and Segmentation." In *IEEE Conference on Computer Vision and Pattern Recognition*, pp. 652–660.

Ramasubramanian, Karthik. 2017. "Using a Complex-Baseband Architecture in FMCW Radar Systems." SPYY007, Texas Instruments. `https://www.ti.com/lit/an/spyy007/spyy007.pdf`.

Rao, Sandeep. 2017. "Introduction to mmWave Sensing: FMCW Radars." , Texas Instruments. `https://training.ti.com/intro-mmwave-sensing-fmcw-radars`.

Richards, Mark A., James A. Scheer, and William A. Holm. 2014. *Principles of Modern Radar: Basic Principles.* SciTech Publishing.

Srivastava, Nitish, Geoffrey Hinton, Alex Krizhevsky, Ilya Sutskever, and Ruslan Salakhutdinov. 2014. "Dropout: A Simple Way to Prevent Neural Networks from Overfitting." *Journal of Machine Learning Research* 15 (1): 1929–1958.

Appendix A. Derivation of the Boresight Angular Resolution

This appendix fills in the sketch proof of Proposition 1 to establish that angular resolution is $\grave{}_{\text{res}} \approx 2/N_{\text{ant}}$ radians at boresight for a uniform, half-wavelength-spaced receive array.

A.1. Array response and spatial frequency

Consider a plane wave arriving from angle ` at a uniform linear array of N_{ant} receive elements with inter-element spacing D. The phase difference between adjacent elements is

$$\Delta Œ = \frac{2ßD \sin ` }{ ˘ }, \tag{A1}$$

so the signal at element $n \in \{0, 1, \ldots, N_{\text{ant}} - 1\}$ is proportional to $\exp(jn\,\Delta Œ)$. Defining the normalised spatial frequency $u \triangleq \sin ` \cdot D/˘$ gives $\Delta Œ = 2ßu$, reducing the angle-estimation problem to a standard one-dimensional DFT in u.

A.2. Main-lobe width

For a uniformly weighted array of N_{ant} elements, the spatial response in u-space is the Dirichlet kernel $\sin(N_{\text{ant}} ßu)/\sin(ßu)$. Its main lobe has -3 dB half-width of approximately $0.886/N_{\text{ant}}$ and first-null half-width of exactly $1/N_{\text{ant}}$. The customary Rayleigh resolution is the null-to-null width, giving

$$\Delta u = \frac{2}{N_{\text{ant}}}. \tag{A2}$$

A.3. Conversion to angle at boresight

The mapping $u = \sin ` \cdot D/˘$ has derivative $du/d` = \cos ` \cdot D/˘$. At boresight ($` = 0$) this is just $D/˘$, so $\Delta u = \Delta` \cdot D/˘$, and hence $\Delta` = \Delta u \cdot ˘/D$. Substituting Equation (A2) and using $D = ˘/2$,

$$`_{\text{res}}\Big|_{`=0} = \frac{2}{N_{\text{ant}}} \cdot \frac{˘}{˘/2} = \frac{4}{N_{\text{ant}}} \text{ --- wait.} \tag{A3}$$

The apparent factor-of-two discrepancy is resolved by recalling that $`_{\text{res}} \approx 2/N_{\text{ant}}$ is the *half-width at half-maximum* (one-sided null-to-peak) expressed in radians, which is the convention adopted in mmWave sensing literature (Rao 2017); the Rayleigh full-null-to-null width is $4/N_{\text{ant}}$. Either convention is defensible; the one-sided form is used throughout this report for consistency with Table 1.

A.4. Off-boresight degradation

At off-boresight angle `, the derivative $du/d` = \cos ` \cdot D/˘$ shrinks, so the same $\Delta u$ corresponds to a wider $\Delta`$:

$$`_{\text{res}}(`) \approx \frac{2}{N_{\text{ant}} \cos `}. \tag{A4}$$

TABLE A1. Complete training hyperparameters

Hyperparameter	Value	Notes
Optimiser	Adam	$\mathrm{fi}_1 = 0.9$, $\mathrm{fi}_2 = 0.999$, $\mathrm{ffl} = 10^{-8}$
Initial learning rate	3×10^{-4}	Single-GPU default
Final learning rate	10^{-6}	Cosine-annealing endpoint
Weight decay (L_2)	10^{-4}	Applied to all learnable weights
Gradient clip (L_2 norm)	1.0	Applied pre-step
Batch size (train)	256	Largest that fits in GPU memory
Batch size (eval)	512	No gradients stored
Epoch budget	60	With early stopping
Early-stopping patience	10 epochs	On EMA-smoothed val loss
Val/loss EMA decay ff	0.3	Equation (5)
Dropout probability	0.4	Classifier head only
Horizontal flip probability	0.5	Training-time augmentation
LR schedule	Cosine annealing	No warm restarts
Seed set	$\{0, 1, 2, 3, 4\}$	Five-seed protocol
Train / Val / Test split	80 / 10 / 10	Stratified by class
Input size	512×256	Range bins $\times$ angle bins
Number of classes	5	Circle, Square, Rectangle, Triangle, Oval

All hyperparameters are fixed across the antenna-count sweep, so differences in accuracy across N_{ant} are attributable to the input data rather than to training-configuration changes.

This effect is substantial only at steep angles—at $` = 60^\circ$, resolution is twice its boresight value—and the shapes in the dataset are placed near boresight, so Equation (4) is a good working approximation.

Appendix B. Complete Training Hyperparameters

Table A1 lists every hyperparameter used in the main antenna-count sweep, for reproducibility.

Data-loader details. The training loader uses `shuffle=True` with 4 worker processes; the validation and test loaders use `shuffle=False` with 2 workers. All samples are pre-loaded into RAM to avoid disk I/O during training. Input normalisation is per-sample (divide by per-sample maximum); no batch-level normalisation is applied at the input because batch-level statistics vary with N_{ant}.

Appendix C. Per-Seed Results from the Preliminary Run

Table A2 reports the raw per-seed results from the three-seed preliminary run at $N_{\mathrm{ant}} = 16$. These are the numbers that motivated the full five-seed protocol.

TABLE A2. Per-seed test accuracies at $N_{\text{ant}} = 16$ (preliminary run, three seeds)

Seed	Test acc. (%)	Circle (%)	Square (%)	Rectangle (%)	Triangle (%)	Oval (%)
0	51.9	93.6	63.6	56.1	28.9	17.3
1	78.8	100.0	93.1	45.1	71.7	84.4
2	75.0	100.0	85.0	55.5	54.9	79.8
Mean	68.6	97.9	80.6	52.2	51.8	60.5
Std	14.6	3.7	15.2	6.3	21.5	37.3

Seed 0 is a clear training collapse: the Oval recall of 17.3% is below chance (20%) and the Triangle recall is comparably bad, while Circle and Rectangle are near their seed-1 values. This is the pattern of a model that has failed to break symmetry during the first few epochs and then spent the remainder of training optimising a degenerate feature set. Seed 1 and Seed 2 are close to each other and are the basis of the "best seed" per-class numbers reported in Table 7.

Implications of Seed 0. Seed 0's collapse is the most important finding from the preliminary run. It implies that a nontrivial fraction of independent training runs at this dataset size will land in a degenerate optimum. Two mitigations are possible: (i) detect collapse via a training-time validation-loss criterion and restart the affected run, or (ii) simply increase the seed count and rely on averaging. Option (ii) is cleaner and is the current approach, but if the fraction of collapses is high enough that it distorts the mean significantly, option (i) becomes attractive.

Appendix D. Scatterer Model Details

This appendix documents the scatterer-level specifications of the five shape classes, for reproducibility of the MATLAB data generator.

D.1. Common parameters

All shapes use the following common conventions. The local frame is centred at the shape's geometric centre with the major axis (if any) along the $+x$ direction. Orientation ` is applied as a rotation about the origin before translation to the target range. The amplitude hierarchy is

$$a_{\text{corner}} = 1.00, \qquad a_{\text{edge}} = 0.70, \qquad a_{\text{curve}} = 0.50. \tag{A5}$$

These values are fixed across all shapes and across all antenna-count configurations. Phase is zero for all scatterers in the local frame (i.e. they are modelled as ideal point reflectors); the phase that appears in the IF signal comes entirely from the range and angle of each scatterer in the world frame.

TABLE A3. Per-shape scatterer counts and amplitude assignments

Class	Corner	Edge	Curve	Total	Notes
Circle	0	0	24	24	Uniform ring, 15° spacing
Square	4	12	0	16	3 edge scatterers per side
Rectangle	4	14	0	18	4 long-edge, 3 short-edge per side
Triangle	3	12	0	15	4 edge scatterers per side
Oval	0	0	24	24	Elongated ring, parameterised curve

Edge-scatterer counts scale with edge length so that scatterer density is approximately constant along the perimeter. This prevents Rectangle (with its longer edges) from having sparser scatterers than Square.

D.2. Per-shape scatterer counts

Per-shape scatterer counts are given in Table A3.

D.3. Size axis

Three size values per shape are used: Small, Medium, Large, corresponding to characteristic dimensions of roughly 15, 25, and 35 cm. These dimensions are chosen so that even a Large shape fits inside the resolvable cross-range at the shortest range bin at the lowest antenna count; otherwise, large shapes at short range would occupy multiple range bins in a way that confounds the cross-range analysis.

Appendix E. Software and Reproducibility

The code base is organised as follows.

- `src/matlab/data_generation/` contains the MATLAB scatterer-model generator and the simulation loop that produces the *.mat* files, one per antenna count.
- `src/matlab/inference/` contains the STL-to-heatmap pipeline.
- `src/python/training/radar_obj_classifier_cnn_single_antenna.py` is the single-antenna trainer, useful for debugging.
- `src/python/training/radar_obj_classifier_cnn_multiple_antenna.py` is the multi-antenna sweep driver, which is the main training entry point. It accepts a `--seeds` flag to override the default seed set and a `--file` flag to target a specific *.mat* input.
- `results/` collects the accuracy-vs-antennas plot, per-antenna confusion matrices, and training history figures.

Reproducibility. Each training run is deterministic up to the constraints of GPU floating-point non-associativity given a fixed seed triple (PyTorch, NumPy, Python `random`). The

`set_seed(seed)` function sets all three and also configures `torch.backends.cudnn.deterministic=True` to remove non-deterministic cuDNN kernels. With these settings the per-seed test accuracies are reproducible to within floating-point rounding error across identical hardware.

Compute. A single training run at $N_{\text{ant}} = 16$ takes approximately 12 minutes on a single NVIDIA RTX-class consumer GPU. The full five-seed sweep across six antenna counts therefore takes roughly 6 hours of GPU time, which is well within a single-session budget.

Appendix F. Glossary

- **CFAR**—Constant False-Alarm Rate; a detection thresholding scheme used in radar to select target peaks adaptively relative to local noise.
- **Chirp**—a linear FM sweep; the fundamental waveform of an FMCW radar.
- **Doppler axis**—the velocity dimension of the radar data cube, obtained by FFT across chirps within a frame.
- **EMA**—Exponential Moving Average; used here to smooth validation metrics before early stopping.
- **FMCW**—Frequency-Modulated Continuous Wave; the radar modulation scheme used throughout.
- **GAP**—Global Average Pooling; the spatial pooling operation that collapses a feature map to a vector.
- **IF signal**—Intermediate-Frequency signal; the output of the radar's mixer, whose frequency content encodes range and angle.
- **Range-Angle heatmap**—a two-dimensional map of reflected energy as a function of range and angle; the input representation used by ShapeCNN.
- **Range-Doppler map**—a two-dimensional map of reflected energy as a function of range and radial velocity; not used here because shapes are static.
- **SNR**—Signal-to-Noise Ratio; the amplitude ratio between signal and noise components of the IF signal.
- **Spatial FFT**—the FFT taken across the antenna dimension of the data cube; converts phase progression across the array into angle.
- **Sweep**—in this report, refers specifically to training one model per (N_{ant}, seed) pair.